

Roteiro Prático: Aplicação de Design Patterns no Simulador de Patos

Grupo: Alex Mendes, Gabriel Madureira, Wendell Leonardo e Yalle Ramos

Objetivo:

. Implementar **padrões de projeto** no simulador de patos para melhorar a estrutura do código e aplicar os conceitos de **POO** de forma prática.

Reflexões Apresentadas pelo Grupo:

. Com o decorrer das discussões feitas pelo grupo decidimos optar pela escolha do padrão de projeto Strategy, pois acreditamos que ele acrescenta mais valor ao nosso código, deixando-o apto para introduzirmos o Princípio Aberto/Fechado do SOLID, levando a um programa aberto para extensão, mas fechado para modificação.

Porque usamos o Strategy?

. Visto que estamos criando um jogo de Patos, e no cenário de um jogo é necessário acrescentarmos novas classes de personagens de tempos em tempos, decidimos utilizar um padrão de projeto que pudesse nos ajudar a adicionar outros tipos de patos com comportamentos diferentes sem que tenhamos que criar subclasses para tudo.

Perguntas para guiar a análise:

.Qual é a função do padrão escolhido?

R: A principal função do padrão de projeto Strategy é permitir que algoritmos ou comportamentos sejam definidos em classes separadas e trocados em tempo de execução.

.Como ele pode ser aplicado ao simulador de patos?

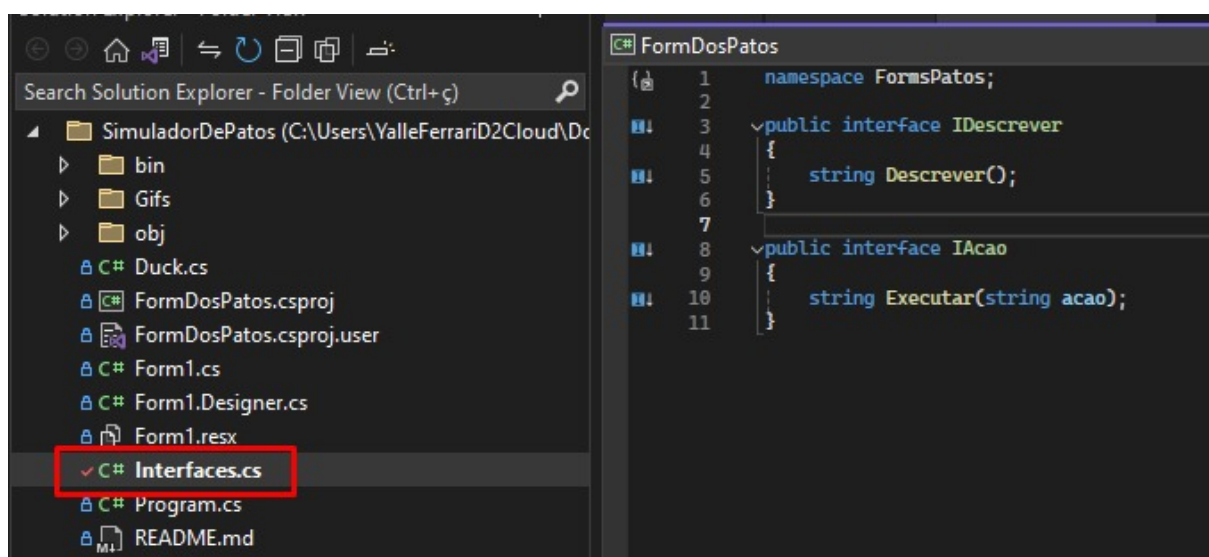
R: O padrão de projeto Strategy pode ser aplicado ao simulador de patos evitando uma herança desnecessária do superclasse Pato.

.Quais classes precisarão ser modificadas ou adicionadas?

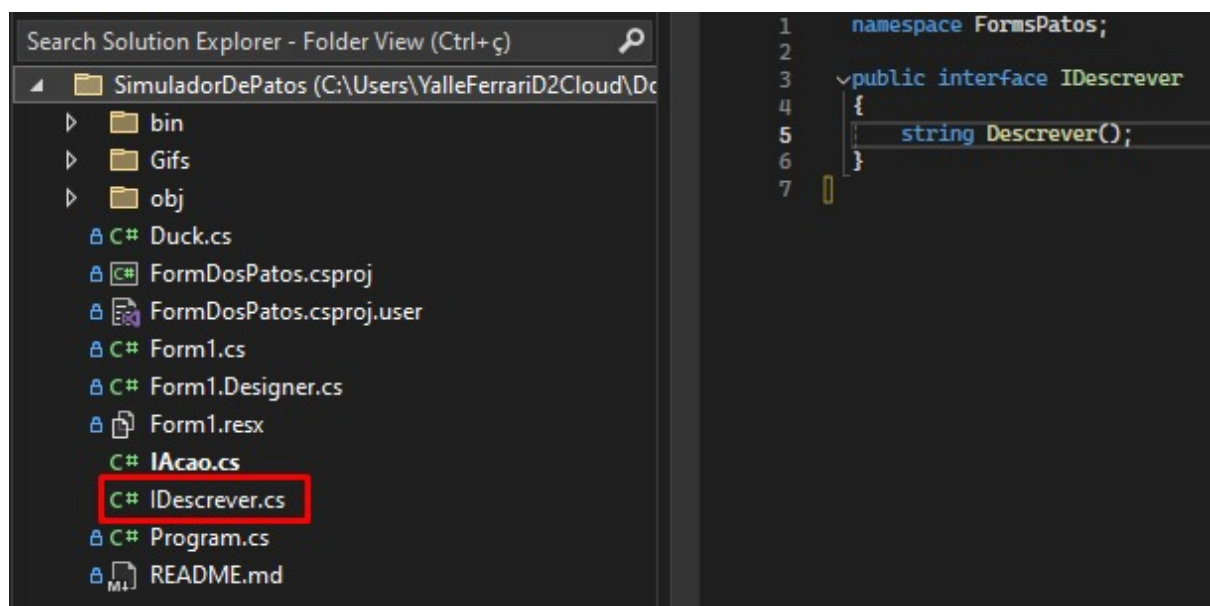
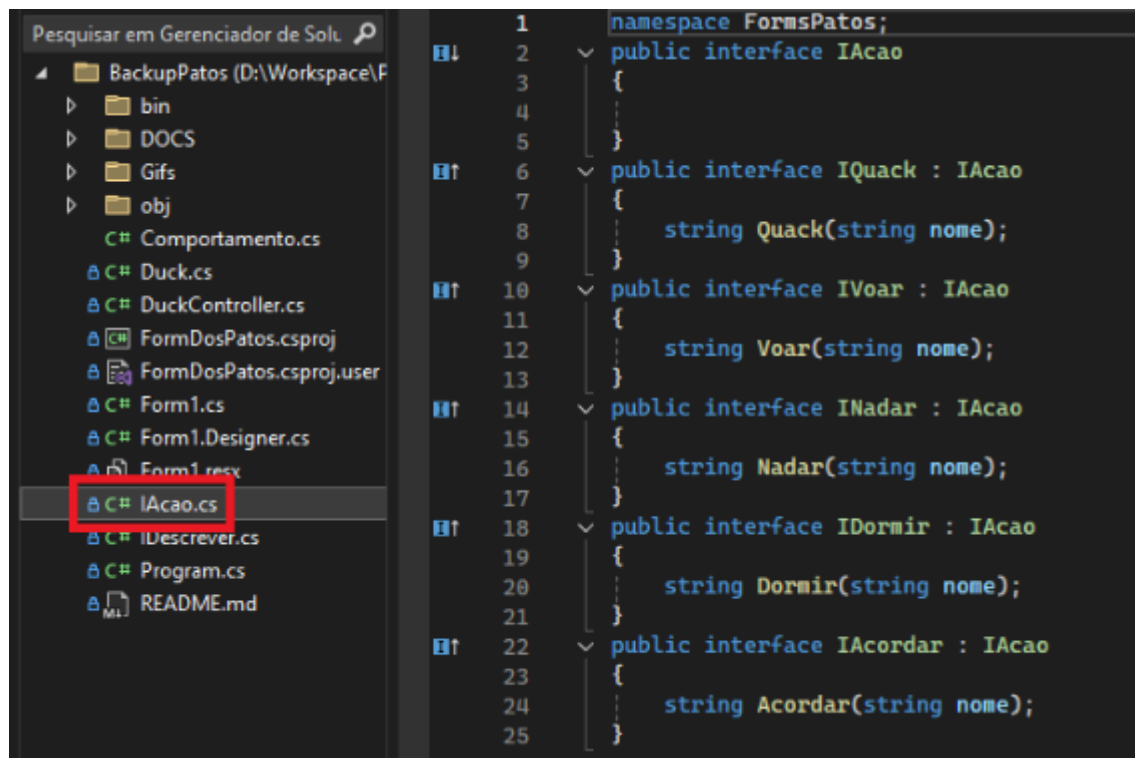
R: As classes que precisarão ser modificadas são a classe “**Form1**”, que deverá ser simplificada, seguindo o **Princípio da Responsabilidade Única (S do SOLID)**, pois está realizando ações a mais do que deveria, sendo assim iremos passar estas ações para a classe “**Duck**”; Outra classe a ser alterada é a “**Interfaces**” que seguirá o **Princípio da Segregação de Interfaces (I do SOLID)**, onde separaremos as ações presentes somente nesta classe em vários outros arquivos .cs, pois **nenhuma classe deve ser forçada a depender de métodos que não utiliza**, e o **Princípio Aberto/Fechado (O do SOLID)**, no qual as classes devem estar **abertas para extensão**, mas **fechadas para modificação**.

Antes e Depois da Classe “Interfaces”

Antes:



Depois:



Antes e Depois da Classe “Duck”

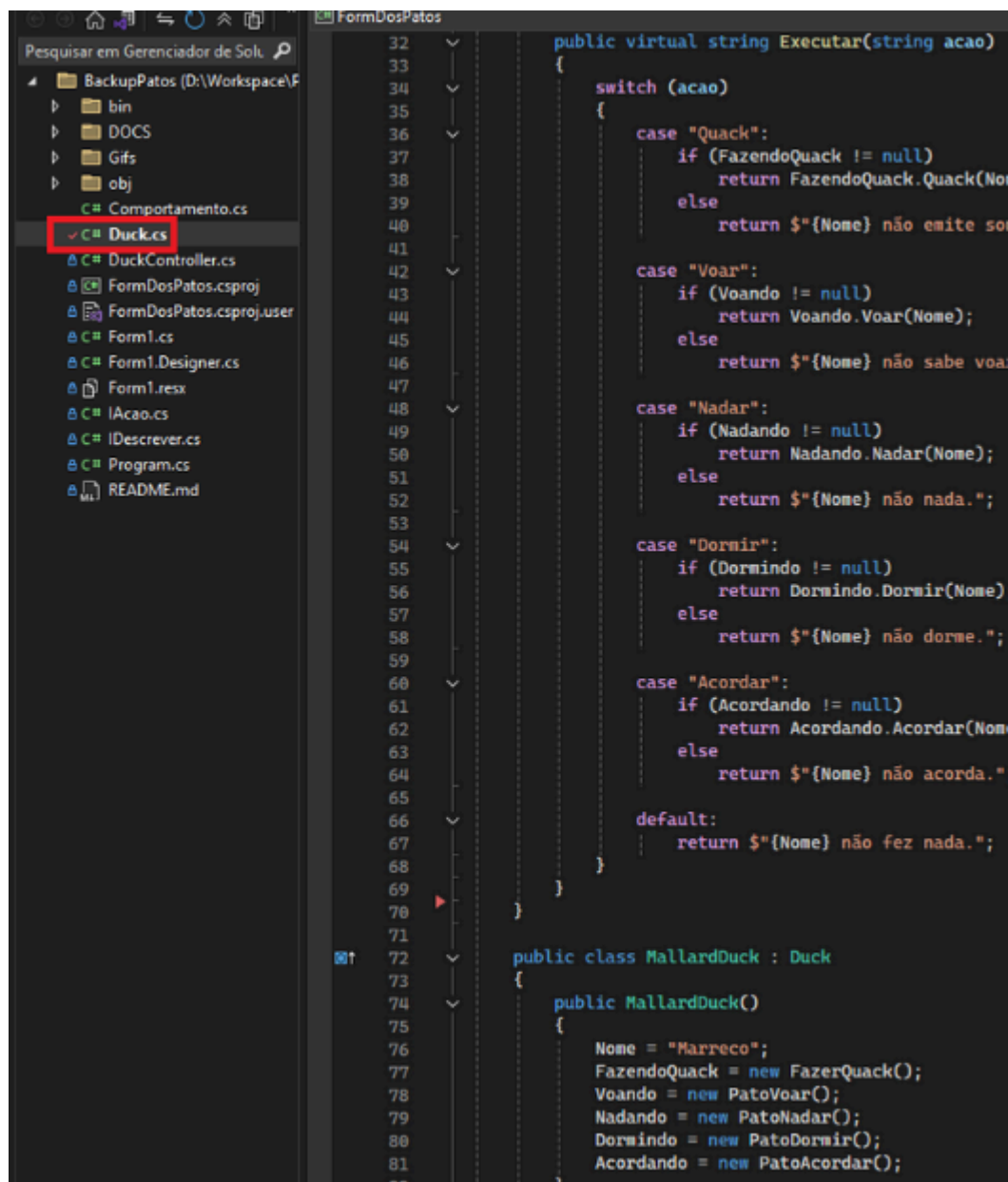
Antes:

```
public virtual string Executar(string acao)
{
    string resultado;

    switch (acao)
    {
        case "Quack":
            resultado = Nome + " está fazendo quack!";
            break;
        case "Voar":
            resultado = Nome + " está voando!";
            break;
        case "Nadar":
            resultado = Nome + " está nadando!";
            break;
        case "Dormir":
            resultado = Nome + " está dormindo!";
            break;
        case "Acordar":
            resultado = Nome + " acordou!";
            break;
        default:
            resultado = Nome + " não fez nada.";
            break;
    }

    return resultado;
}
```

Depois:



Antes e Depois da Classe “DuckController”

Antes:

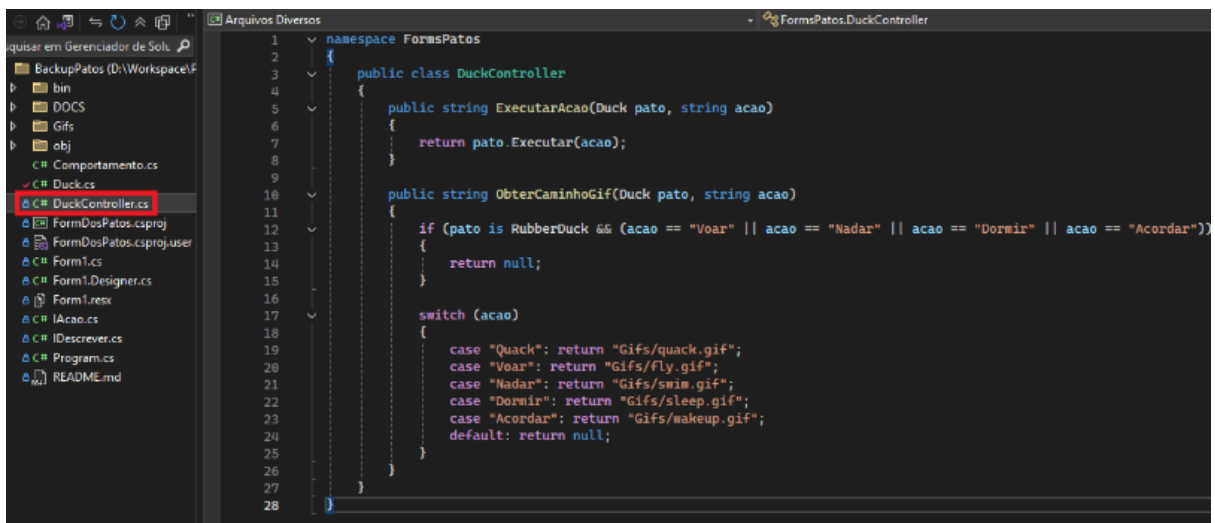
```

public string ObterCaminhoGif(Duck pato, string acao)
{
    if (pato is RubberDuck && (acao == "Voar" || acao == "Dormir" || acao == "Acordar"))
    {
        return null;
    }

    if (acao == "Quack")
        return "Gifs/quack.gif";
    else if (acao == "Voar")
        return "Gifs/fly.gif";
    else if (acao == "Nadar")
        return "Gifs/swim.gif";
    else if (acao == "Dormir")
        return "Gifs/sleep.gif";
    else if (acao == "Acordar")
        return "Gifs/wakeup.gif";
    else
        return null;
}

```

Depois:



```

1 namespace FormsPatos
2 {
3     public class DuckController
4     {
5         public string ExecutarAcao(Duck pato, string acao)
6         {
7             return pato.Executar(acao);
8         }
9
10        public string ObterCaminhoGif(Duck pato, string acao)
11        {
12            if (pato is RubberDuck && (acao == "Voar" || acao == "Nadar" || acao == "Dormir" || acao == "Acordar"))
13            {
14                return null;
15            }
16
17            switch (acao)
18            {
19                case "Quack": return "Gifs/quack.gif";
20                case "Voar": return "Gifs/fly.gif";
21                case "Nadar": return "Gifs/swim.gif";
22                case "Dormir": return "Gifs/sleep.gif";
23                case "Acordar": return "Gifs/wakeup.gif";
24                default: return null;
25            }
26        }
27    }
28 }

```

Testes e Ajustes

Guias para teste:

.O código reflete o padrão corretamente?

R: O código está seguindo corretamente o padrão de projeto Strategy Pattern, além de estar alinhado aos Princípios da Responsabilidade Única, Aberto/Fechado e Segregação de Interfaces do SOLID.

.A nova funcionalidade funciona dentro do simulador?

R: As novas funcionalidades aplicadas estão funcionando de forma esperada dentro do simulador.

.O código ficou mais organizado e reutilizável?

R: O código após as alterações ficou mais organizado e legível, agora podendo ser reutilizado com mais facilidade.